Chapter 12. Profiling

The term *profiling* has a somewhat loose usage among programmers. There are, in fact, several different approaches to profiling that are possible, the two most common of which are:

- Execution
- Allocation

In this chapter, we will cover both of these topics. Our initial focus will be on execution profiling, and we will use this subject to introduce the tools that are available to profile applications. Later in the chapter, we will introduce memory profiling and see how the various tools provide this capability.

One of the key themes we will explore is just how important it is for Java developers and performance engineers to understand how profilers operate in general. Profilers are very capable of misrepresenting application behavior and exhibiting noticeable biases.

Execution profiling is one of the areas of performance analysis where these biases come to the fore. The cautious performance engineer will be aware of this possibility and will compensate for it in various ways, including profiling with multiple tools to understand what's really going on.

It is equally important for engineers to address their own cognitive biases, and to not go looking for the performance behavior that they expect. The antipatterns and cognitive traps that we met in Chapter 2 (and see also Appendix B) are a good place to start when training ourselves to avoid these problems.

Introduction to Profiling

In general, JVM profiling and monitoring tools operate by using some low-level instrumentation and either streaming data to an external tool (sometimes a GUI graphical console or a SaaS) or saving it in a log for later analysis. The low-level instrumentation often takes the form of either an agent loaded at application start or a component that dynamically attaches to a running JVM.

NOTE

Agents were introduced in "Monitoring and Tooling for the JVM"; they are a very general technique with wide applicability in the Java tooling space.

In broad terms, we need to distinguish between *monitoring tools* (whose primary goal is observing the system and its current state), *alerting systems* (for detecting abnormal or anomalous behavior), and *profilers* (which provide deep-dive information about running applications). These tools have different, although often related, objectives and a well-run production application can use all of them.

The focus of this chapter, however, is profiling, the aim of which is to identify user-written code that is a target for refactoring and performance optimization.

As discussed in "A Simple System Model", the first step in diagnosing and correcting a performance problem is to identify which resource is causing the issue. An incorrect identification at this step can prove very costly.

> *The scary thing about benchmarks is that they always produce a number, even if that number is meaningless. They measure something; we're just not sure what.*
>
> —Brian Goetz, "Anatomy of a flawed microbenchmark"

In other words, profiling tools will always produce a number—it's just not clear that the number has any relevance to the problem being addressed. For this reason we introduced some of the main types of bias in Chapter 2 and delayed discussion of profiling techniques until now.

> *A good programmer...will be wise to look carefully at the critical code; but only after that code has been identified.*[1]
>
> —Donald Knuth

This means that before undertaking a profiling exercise, performance engineers should have already identified a performance problem. This identification can come from a number of sources, including:

- Performance regression tests in dev or CI pipelines
- UAT or dedicated performance testing environments
- Changes in production—for example, by observing the behavior of a canary

- Originally acceptable performance that has now become a problem—for example, by running out of capacity or data growth, exposing insufficient indexing

Note that performance regression tests can be difficult to write well, and most applications should structure them in the form of integration tests rather than microbenchmarks.

Once a performance problem has been identified (by whatever route), then the next step is to figure out what has caused it. It might be the case that application code is to blame, but it could also be something such as a library dependency upgrade that has brought in a performance regression. If you have performance regression tests as part of CI/CD, then you will want the deployment to fail and prevent that from going to Production.

In general, if the application is consuming close to 100% of CPU in user mode (which we can detect via metrics or alerts), then this is strong evidence for a performance problem that should be tackled with execution profiling. However, we must also remember that—even if the CPU is fully maxed out in user mode (not kernel time)—there is another possible cause that must be ruled out before profiling: garbage collection.

All applications that are serious about performance should be logging GC events, so this check is a simple one: consult the GC log and application logs for the machine and ensure that the GC log is quiet and the application log shows activity. If the GC log is the active one, then GC tuning should be the next step—not execution profiling.

Maxed-out CPU is not the only situation where an execution profiling can be useful, however. For example, if the application is not meeting latency SLAs in Production, you might choose to see what the profiler can tell you. If it's got high lock contention (which a profiler will tell you), then that will prevent it from using all the available cores.

As a second example, an application that's blocking on database I/O because a code change has put in a new, and expensive, `SELECT` query would also show up in an execution profiling run.[2] Some problems will only show up in profiling data from Production, however. This can occur when Staging does not have enough data in it for the missing index or expensive `SELECT` to hurt badly enough, but Production definitely does.

GUI Profiling Tools

In this section, we will discuss two different profiling tools with graphical UIs. There are quite a few tools available in the market, so we focus on two of the most common OSS tools rather than attempting an exhaustive survey. Our focus will be on execution profiling here, but the tools do offer a variety of other capabilities as well.

VisualVM

As a first example of a profiling tool, let's consider the VisualVM, which we met in "Monitoring and Tooling for the JVM". It includes both an execution and a memory profiler and is a very straightforward free profiling tool.

It is quite limited—it is rarely usable as a production profiler, but it can be helpful to performance engineers who want to understand the behavior of their applications in dev and QA environments.

We have already met some of the common screens in VisualVM—but let's consider the Monitor tab again. This shows basic telemetry information, as we can see in Figure 12-1, and is often the starting point for a profiling investigation.

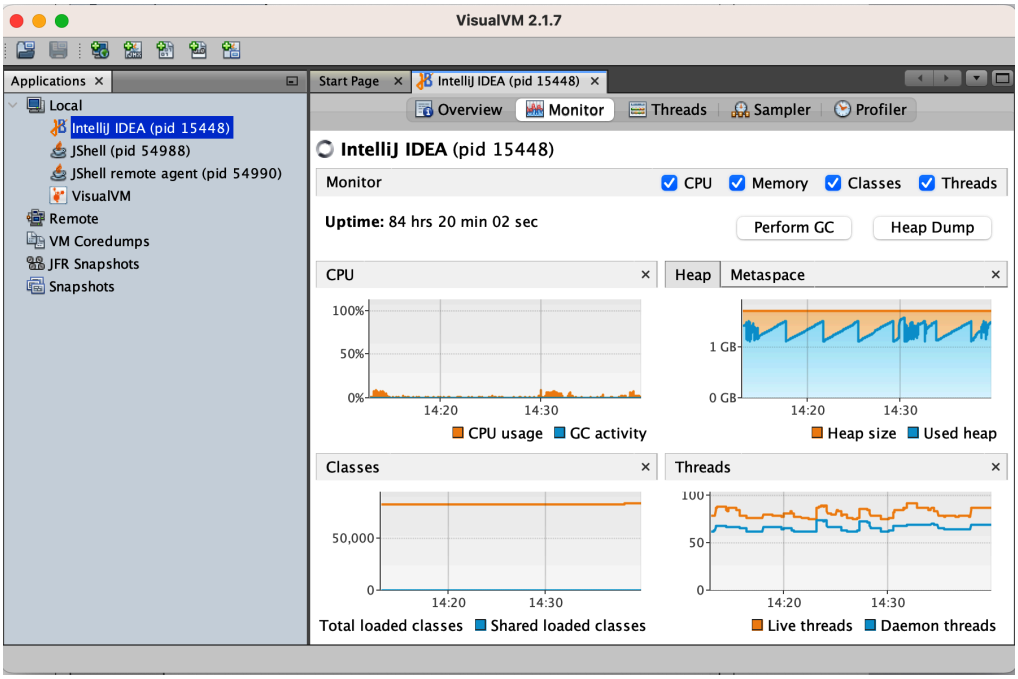

Figure 12-1. VisualVM monitor view

In Figure 12-2 we can see the execution profiling view of VisualVM from the Profiler tab.

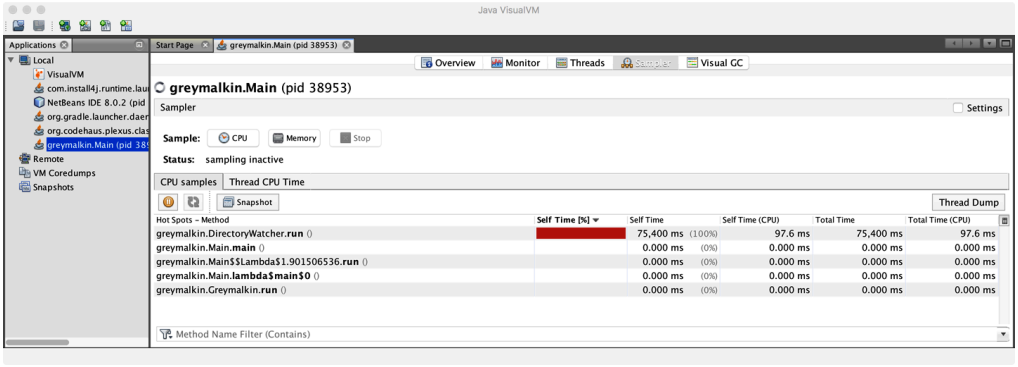

Figure 12-2. VisualVM execution profiler

This shows a simple view of executing methods and their relative CPU consumption. It can be a useful first tool for performance engineers who are new to the art of—and tradeoffs involved in—profiling. However, the amount of profiling drill-down that is possible within VisualVM's UI is really quite limited, and most performance engineers quickly outgrow it and turn to one of the more complete tools on the market.

JDK Mission Control

The JDK Flight Recorder and JDK Mission Control tools (known as JFR/JMC) are profiling and monitoring technologies that Oracle obtained as part of its acquisition of BEA Systems.

The two technologies are separate but related:

- JFR is a low-level, event-based, performance data collection capability that is embedded into the HotSpot JVM, and it provides events for the OS, the JVM, and JDK libraries.[3]
- JMC is the graphical component, and its initial installation consists of a JMX Console and a handler for JFR data, although more plug-ins can easily be installed from within Mission Control.

These tools were originally part of the tooling offering for BEA's JRockit JVM and were moved to the commercial version of Oracle JDK as part of the process of retiring JRockit. To support the port from JRockit, the HotSpot VM was instrumented to introduce a large basket of additional performance counters.

When Java 11 was released, JFR was donated to OpenJDK, and JMC was moved to a standalone project. Subsequently, JFR was backported to OpenJDK 8 as part of Update 272—this means that recent releases of OpenJDK 8 have JFR available.

NOTE

The JMC desktop application can be built from source or downloaded from several places, including the Eclipse Adoptium project.

JMC is started up by running the `jmc` binary. The startup screen for Mission Control can be seen in Figure 12-3.

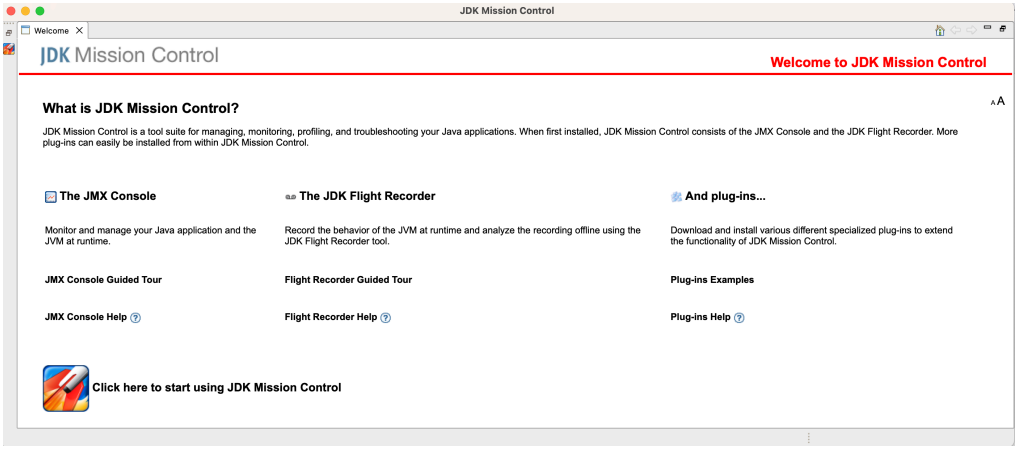

Figure 12-3. JMC startup screen

To profile, Flight Recorder must be enabled on the target application. You can achieve this in one of three ways: by starting the application with the JFR flags enabled or by dynamically attaching after the application has already started, or by starting the application and then using the `jcmd` command to start a JFR recording in a JVM on the local machine.[4]

Once it is attached, enter the configuration for the recording session and the profiling events, as shown in Figure 12-4.

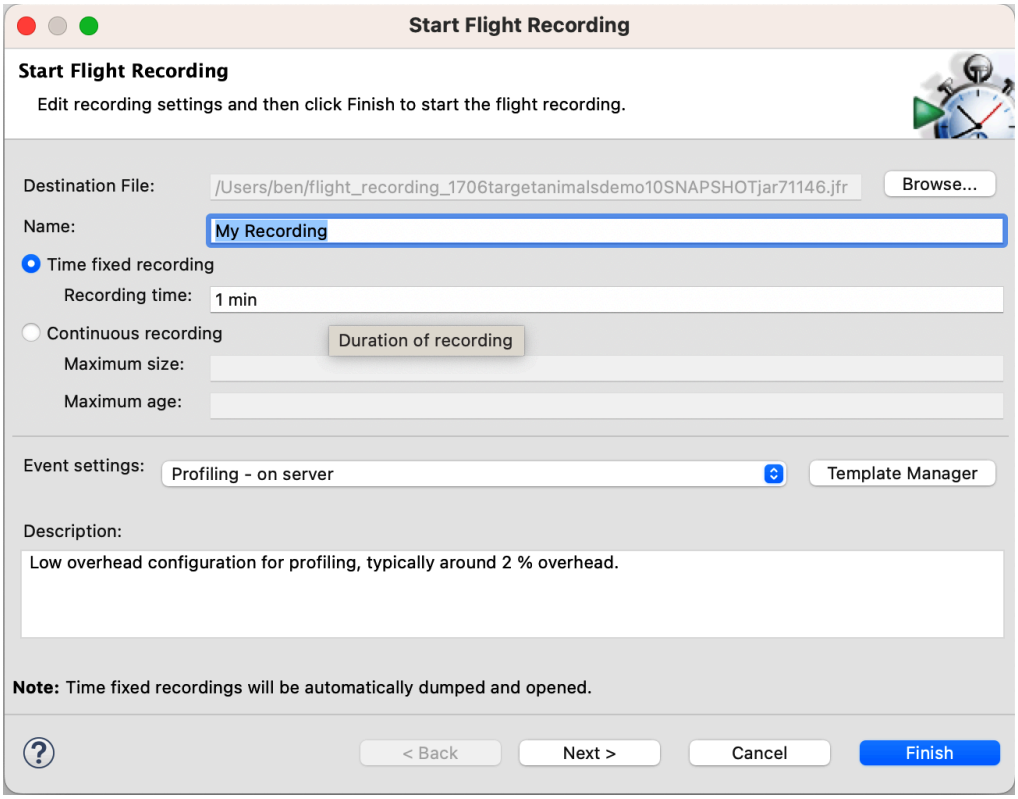

Figure 12-4. JMC recording setup

As discussed earlier in the chapter, execution profiling is very much not a silver bullet, and engineers must proceed carefully to avoid confusion and misconceptions. There are inevitable performance tradeoffs in the configuration of the tool, as well as a need for an awareness of the overhead of profiling.

NOTE

These screenshots depict the case of an application that is idle—i.e., not performing much (if any) work, so the images are for demonstration purposes only.

When the recording completes, an automated analysis is displayed in the main window, with the lefthand side showing a wide variety of available events. It looks like the view shown in Figure 12-5.

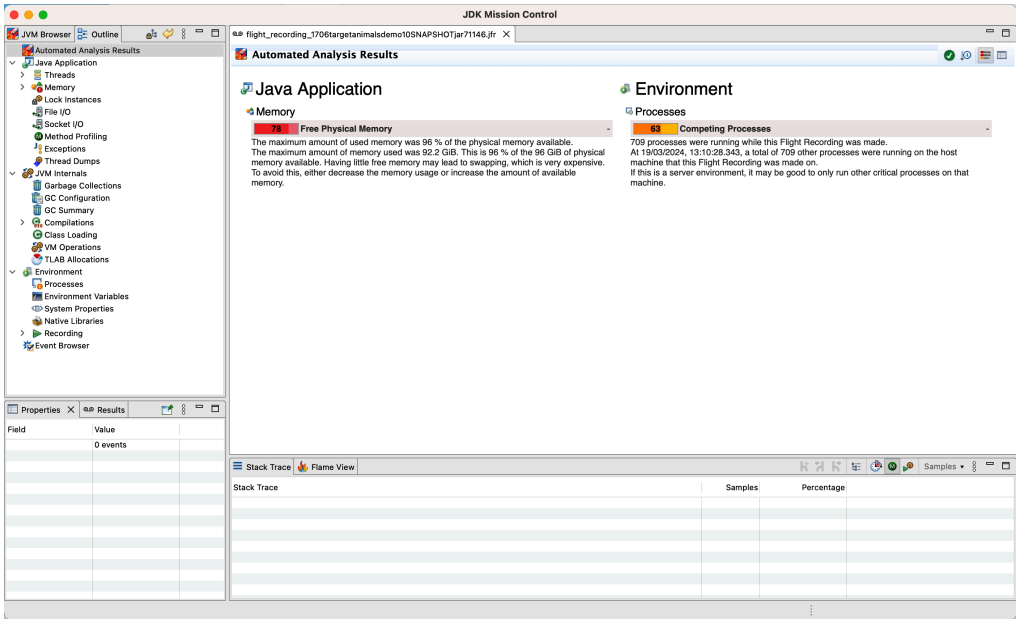

Figure 12-5. JMC automated analysis

Let's start by selecting the Method Profiling entry from the lefthand navigation. This results in a screen like Figure 12-6.

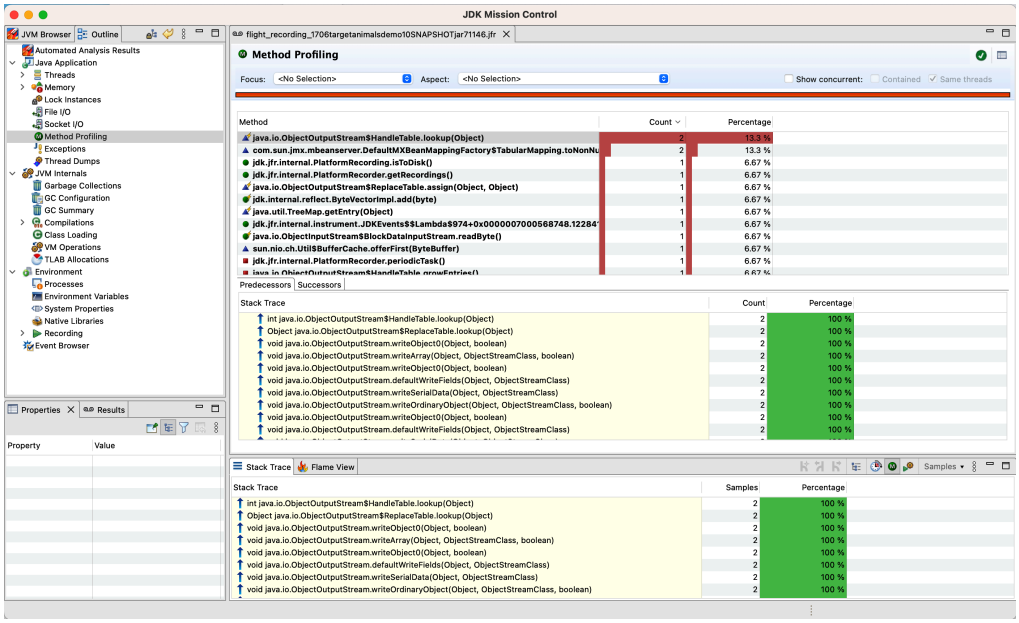

Figure 12-6. JMC Method Profiling

From here, we can start to look at which of the hot methods are occupying the majority of the application runtime and can also use techniques such as flame graphs (which we will discuss in more detail later)—provided that our profiling setup supports them.

Overall, JMC is a solid GUI-based profiling tool—but there are limits to this type of tooling. In particular, it can be difficult to see how to apply desktop-based profiling to the sorts of observability techniques we met in Chapters 10 and 11.

We will see how to bridge this gap presently, but first we need to discuss some details of how execution profilers actually operate.

Sampling and Safepointing Bias

Traditionally, execution profiling uses periodic sampling of stack traces to obtain a view of what code is running on each thread. This is because it is not cost-free to take measurements, so tracking all method entries and exits would represent an excessive data collection cost. So, instead, a sampled snapshot is taken—but this can only be done at a relatively low frequency without unacceptable overhead.

For example, the thread profiler in the New Relic Java agent will sample every 100 ms—and this is often considered a rule of thumb, or best guess, of a limit on how often samples can be taken without incurring unacceptably high overhead.

In other words, the sampling interval represents a tradeoff for the performance engineer. Sample too frequently, and the overhead becomes unacceptable, especially for an application that cares about performance. On the other hand, sample too infrequently, and the chance of missing important behavior becomes too large, as the sampling may not reflect the real performance of the application.

> *By the time you're using a profiler it should be filling in detail—it shouldn't be surprising you.*
>
> —Kirk Pepperdine (personal correspondence)

Not only does sampling offer opportunities for problems to hide in the data, but in many profilers, sampling has only been performed at JVM safepoints. This is known as *safepointing bias* and has two primary consequences:

- All threads must reach a safepoint before a sample can be taken.
- The sample can only be of an application state that is at a safepoint.

The first of these imposes additional overhead on producing a profiling sample from a running process. The second consequence skews the distribution of sample points by sampling only the state when it is already known to be at a safepoint.

NOTE

Knowing that a profile is only showing running code because of the way it samples is crucial, because blocking and waiting are as large a headache as inefficient algorithms.

Sampling execution profilers use the `GetCallTrace()` function from HotSpot's C++ API to collect stack samples for each application thread. The usual design is to collect the samples within an agent, and then log the data or perform other downstream processing.

However, in its original implementation, `GetCallTrace()` had a quite severe overhead: if there are *N* active application threads, then collecting a stack sample caused the JVM to safepoint *N* times. This overhead is one of the root causes that set an upper limit on the frequency with which samples can be taken, at least for Java 8 and before.

This limitation was, to some extent, addressed by JEP 312, "Thread-Local Handshakes", which was also necessary groundwork for the Shenandoah and ZGC collectors that we discussed in Chapter 5. More recent versions, from Java 11 onward, have a significantly reduced overhead for thread profiling as a result of this change.

In general, the careful performance engineer will, therefore, keep an eye on how much safepointing time is being used by the application. If too much time is spent in safepointing, then the application performance will suffer, and any tuning exercise may be acting on inaccurate data. A JVM flag that can be very useful for tracking down cases of high safepointing time is:

```
-XX:+PrintGCApplicationStoppedTime
```

This will write extra information about safepointing time into the GC log. Some tools can automatically detect problems from the data produced by this flag and differentiate between safepointing time and pause time imposed by the OS kernel.

One example of the problems caused by safepointing bias can be illustrated by a *counted loop*. This is a simple loop, of a similar form to this snippet:

```
for (int i = 0; i < LIMIT; i += 1) {
    // only "simple" operations in the loop body
}
```

We have deliberately not defined the meaning of a "simple" operation in this example, as the behavior depends on the exact optimizations that the JIT compiler can perform. Further relevant details can be found in "Diagnosing Application Problems Using Observability".

Examples of simple operations include arithmetic operations on primitives and method calls that have been fully inlined (so that no methods are actually within the body of the loop).

If `LIMIT` is large, then the JIT compiler will translate this Java code directly into an equivalent compiled form, including a back branch to return to the top of the loop. As discussed in "JVM Safepoints", the JIT compiler inserts safepoint checks at loop-back edges. This means that for a large loop, there will be an opportunity to safepoint once per loop iteration.

However, for a small enough `LIMIT` this will not occur, and instead the JIT compiler will unroll this loop. This means that the thread executing the small-enough counted loop will not safepoint until after the loop has completed.

Only sampling at safepoints has, thus, led directly to a biasing behavior that is sensitively dependent on the size of the loops and the nature of operations that we perform in them.

This is obviously not ideal for rigorous and reliable performance results. Nor is this a theoretical concern—loop unrolling can generate significant amounts of code, leading to long chunks of code where no samples will ever be collected.

However, there is an alternative to sampling profilers, and it's the subject of our next section.

Modern Profilers

In this section, we will discuss three modern open source tools that can provide better insight and more accurate performance numbers than traditional sampling profilers. These tools are:

- perf
- Async Profiler
- Honest Profiler

We'll discuss each in turn—let's start with the perf tool.

perf

perf is a useful lightweight profiling solution for applications that run on Linux. It is not specific to Java/JVM applications but instead reads hard-

ware performance counters and is included in the Linux kernel, under `tools/perf`.

Performance counters are physical registers that count hardware events of interest to performance analysts. These include instructions executed, cache misses, and branch mispredictions. This forms a basis for profiling applications.

Java presents some additional challenges for perf, due to the dynamic nature of the Java runtime environment. To use perf with JVM applications, we also need a bridge to handle mapping the dynamic parts of VM execution.

This bridge is perf-map-agent, an agent that will generate dynamic symbols for perf from unknown memory regions (including JIT-compiled methods). Due to HotSpot's dynamically created interpreter and jump tables for virtual dispatch, these must also have entries generated. perf-map-agent consists of an agent written in C and a small Java bootstrap that attaches the agent to a running Java process if needed.

In Java 8u60, a new flag was added to enable better interaction with perf:

```
-XX:+PreserveFramePointer
```

Unfortunately, this flag defaults to `false`, so when using perf to profile Java applications, it is strongly advised that you explicitly switch it on.

NOTE

Activating this flag disables a JIT compiler optimization, so it can decrease performance slightly.

One striking visualization of the numbers perf produces is the flame graph. This shows a highly detailed breakdown of exactly where execution time is being spent. An example can be seen in Figure 12-7.

The flame graph technique has evolved over time, so there are some important details about reading a flame graph that you should be aware of:

- The *x*-axis shows the stack profile population, sorted alphabetically.
- The *y*-axis shows stack depth, counting from the bottom.
- Each rectangle represents a stack frame, and the wider a frame is, the more often it was present in the stacks.

- The top rectangle shows what is on a specific CPU, and beneath it is its ancestry.
- Originally, flame graphs used random colors to help visually differentiate adjacent frames.

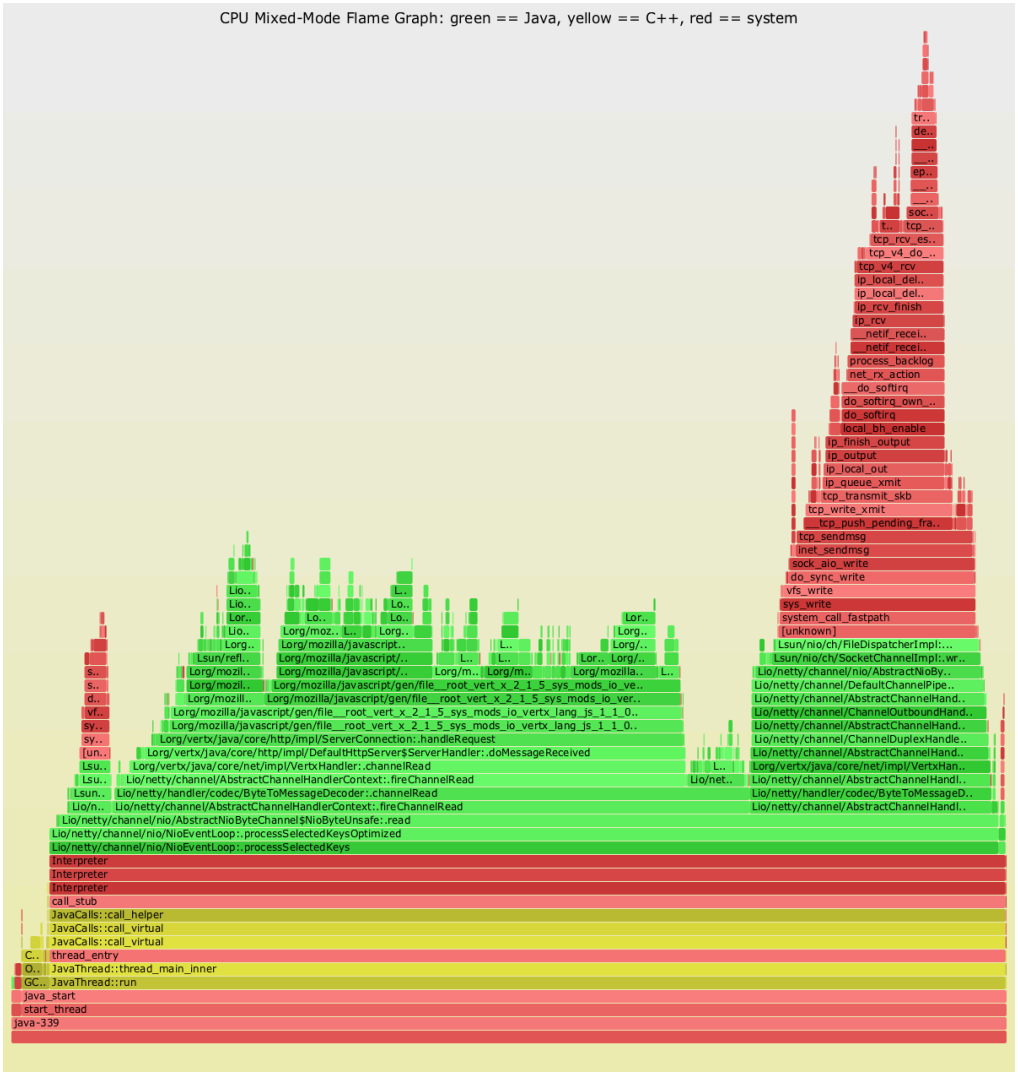

Figure 12-7. Java flame graph

Of these points, understanding that the *x*-axis is not the passage of time is perhaps the most important. Flame graphs organize the samples alphabetically and merge frames wherever possible—this gives a better overview of the big picture of the profile.

There have been several enhancements and variations on Gregg's original concept of flame graphs. For example, some implementations have begun using a consistent coloring scheme, in which the color of a rectangle has some semantic meaning (e.g., if it's a Java or a native frame), rather than just using the colors to make the graphs easy to read.

One important variation is *flame charts*, which were originally developed for Google Chrome's WebKit Web Inspector. Flame charts have time on the *x*-axis instead of the alphabetical sorting. This has the advantage that time-based patterns can be displayed but comes at the expense of only being able to sensibly display the pattern for a single thread. This is often

fine for JavaScript applications, which are typically single-threaded, but is much less useful for Java applications.

The Netflix Technology Blog has some detailed coverage of how the team has implemented flame graphs on its JVMs.

You should also be aware of some issues with running perf in containers--remember that perf is based on hardware performance counters. The events that perf relies upon for CPU profiling may not be available in containers (or in seccomp environments).

Let's move on to look at Async Profiler, which uses perf as a building block for JVM profiling.

Async Profiler

The key goals of Async Profiler are to:

- Remove the safepointing bias that most other profilers have.
- Operate with significantly lower overhead than traditional profilers.

To achieve this, Async Profiler uses a private API call: `AsyncGetCallTrace` (or AGCT) within HotSpot. This means, of course, that Async Profiler will not work on a non-OpenJDK JVM. It will work with OpenJDK-based JVMs (including builds from Adoptium, Amazon, Microsoft, Oracle, Red Hat, and Zulu from Azul) as well as HotSpot JVMs that have been built from scratch.

NOTE

In general, tools like Async Profiler are more commonly run in headless mode as data collection tools. In this approach, visualization is provided by other tooling or custom scripts.

Its reliance on perf means that Async Profiler also works only on the operating systems where perf works (primarily Linux). The implementation of Async Profiler uses the Unix OS signal `SIGPROF` to interrupt a running thread. The call stack can then be collected via the private `AsyncGetCallTrace()` method.

This only interrupts threads individually, so there is never any kind of global synchronization event. This avoids the contention and overhead typically seen in traditional sampling profilers. Within the asynchronous callback, the call trace is written into a lock-free ring buffer. A dedicated

separate thread then writes out the details to a log without pausing the application.

There are some additional details about the operation of nonsampling profilers that you need to be aware of. First of all, let's consider this question:

When a CPU receives an "externally triggered" interrupt (e.g., a cache miss), at what point in the instruction stream is the CPU interrupted?

This question is especially important for modern out-of-order (OOO) CPUs, which is essentially everything a modern serverside application is likely to be running on.

If the output of a low-level profiler (such as perf) is examined, then it is possible to see an instruction tagged with, say, an L1 cache miss event that was not caused by this instruction. This is known as *skid* and is defined as the distance between the instruction that actually caused the event and the instruction where the event is tagged.

A related problem (specific to the JVM) is that there is also a hidden form of safepoint bias. Even though nonsampling profilers using AGCT or a similar technique are not safepoint biased for collecting stacktraces, the resolution of the last frame is still biased toward the recorded debug information, and unfortunately, by default, these are at safepoints.

To counteract this, nonsampling profilers try to activate the flag `-XX:+DebugNonSafepoints` as soon as possible to have more precise resolution.

Async Profiler also tries to solve the `perf_events` issue that affects containers--by adding a new CPU sampling engine based on `timer_create` instead. This combines benefits of the older sampling engines, with the small tradeoff that it cannot collect kernel stacks.

This means that recent versions of Async Profiler work in containers by default, have reduced timing biases, and also do not consume file descriptors.[5]

TIP

To use Async Profiler standalone, a certain amount of ceremony and scripting specific to your application is required. Rather than delve into those complexities, external resources, such as the Async Profiler GitHub page should be consulted.

Finally, an alternative choice to Async Profiler that was popular a few years ago is Honest Profiler. It uses the same internal API as Async Profiler, and is also an open source tool that runs only on HotSpot JVMs. However, it does not seem to be actively maintained any longer, so it should not be used for new projects (and any existing installations of it should be migrated away from).

In the next section, we'll examine JFR—a built-in profiler that is, in many ways, similar to Async Profiler.

JDK Flight Recorder (JFR)

We met JFR in "GUI Profiling Tools"—it is a low-overhead tool shipped as part of OpenJDK for gathering diagnostics and profiling data. The intent is that it should be used by an in-flight Java application running on the HotSpot JVM in production.

For a production profiler, the overhead needs to be small enough to be tolerated both during normal and high-usage scenarios. JFR achieves this with the use of *profiles*—remember that JFR is event-based, so the different profiles correspond to different sets of events being enabled for JFR to respond to.

Out of the box, JFR ships with two profiles: "Continuous" (sometimes called "Default") and "Profiling." The XML config files corresponding to these profiles are present in the JDK installation as `default.jfc` and `profile.jfc`.

Of these two stock profiles, Continuous is designed for always-on profiling but may lack important detail, especially for allocation profiling. The "Profiling" profile (a truly classic example of why engineers shouldn't be allowed to name things) has much more detail but also a higher runtime overhead.

NOTE

Advanced users of JFR can choose to create a custom profile, containing a different set of events, which better reflects the performance interests of the group maintaining the application.

On the subject of overhead—according to Oracle presentations and demos—JFR profiling has about ~1% impact to steady state application performance for the Continuous profile. The authors are in broad agree-

ment—having generally observed impact in the ~3% range for a profile that includes allocation profiling.

Alternatively, a more systematic study (but one conducted on microbenchmarks rather than full systems) can be found in "Don't Trust Your Profiler: An Empirical Study on the Precision and Accuracy of Java Profilers"[6].

This paper broadly follows the methodology of Georges et al. (2007), which we briefly discussed in Chapter 2, and compares JFR and several other profilers (including Async Profiler and Honest Profiler, which we have already met).

This consensus is that, in general, JFR induces an overhead that lies within the acceptable range for profilers, and that it is possible to use it to perform always-on profiling (although some applications may have resource requirements that make the overhead unaffordable).

Due to this, JFR has been used as a foundation to build many other observability and monitoring tools. For example, both Datadog and New Relic ship execution profilers that use JFR data as input data.

The key to understanding what's possible with JFR is the events, and what data can be found in them. So, let's take a closer look.

JFR events are typed data, and each event type has a name and a structure. For example, the event `jdk.CPULoad` represents a metric time series of CPU data, with several fields such as `jvmUser`, `jvmSystem`, and `machineTotal` as well as a timestamp represented by the event start time.

Other events have different structures—such as the GC events, which are fine-grained and may correspond to just a single phase of a collection cycle. Or the lock events, such as `jdk.JavaMonitorEnter`, which have a threshold value, enabling JFR to record only those instances where a Java monitor was held for longer than a specified time (e.g., 10 ms).

The JDK ships with a simple command-line tool that you can use to analyze a JFR dump file. These dump files can be created in one of three ways—using the JMC GUI, or via the command-line `jcmd` to control a running Java process, or by adding a suitable command-line switch to your Java startup.

For the `jcmd` route, we need to execute three separate commands to generate a file, one to start, one to dump, and then one to shut down the

recording operation once we've collected enough data:

```
jcmd <pid> JFR.start name=MyRecording settings=default
jcmd <pid> JFR.dump filename=my-recording.jfr
jcmd <pid> JFR.stop
```

To begin a JFR recording at startup, we need to add this command-line switch and provide suitable options:

```
-XX:StartFlightRecording:<options>
```

These recordings can be of a fixed duration or use a ring-buffer mode (as we will discuss later in the chapter). Once we've obtained a dump file, we can use the `jfr` tool that the JDK also provides to look at the events in it.

NOTE

The `jfr` tool has many subcommands—use `jfr --help` to see them all.

For example, we can see the `CPULoad` and `JavaMonitorEnter` events with a single `jfr print` command like this:

```
jfr print --events CPULoad,JavaMonitorEnter recording.jfr
```

which can produce output similar to this:

```
...

jdk.CPULoad {
  startTime = 11:51:57.745
  jvmUser = 8.75%
  jvmSystem = 0.57%
  machineTotal = 13.50%
}

jdk.JavaMonitorEnter {
  startTime = 11:51:58.065
  duration = 12.1 ms
  monitorClass = jdk.jfr.internal.PlatformRecorder (classLoader = bootst
  previousOwner = "RMI TCP Connection(idle)" (javaThreadId = 32)
  address = 0x12CE66508
  eventThread = "JFR Periodic Tasks" (javaThreadId = 26)
}
```

```
...
```

By experimenting with a few `jfr print` commands, you can see the different shapes of the different event types. The `jfr` tool can also produce output in XML and JSON formats.

TIP

An event browser for JFR events is maintained by the team at SAP and covers LTS and recent feature releases of OpenJDK.

It is also relatively simple to handle JFR dump files programmatically, as it's a simple matter of looping through a `RecordingFile` object:

```
String fileName = // ... some JFR file
var recording = new RecordingFile(Paths.get(fileName));
while (recording.hasMoreEvents()) {
    var event = recording.readEvent();
    if (event != null) {
        var details = decodeEvent(event);
        if (details == null) {
            System.err.println("Failed to recognize details");
        } else {
            // We'd process details here, for now just log
            System.out.println(details);
        }
    }
}
```

Of course, the events of interest still need to be decoded in the `decodeEvent()` method. One way to do this is to use a static collection of mappers, which are applied like this:

```
public Map<String, String> decodeEvent(final RecordedEvent e) {
    for (var ent : mappers.entrySet()) {
        if (ent.getKey().test(e)) {
            return ent.getValue().apply(e);
        }
    }
    return null;
}

private static Predicate<RecordedEvent> makePredicate(String s) {
    return e -> e.getEventType().getName().startsWith(s);
```

```
}

private static final Map<Predicate<RecordedEvent>,
            Function<RecordedEvent, Map<String, String>>> mappers =
    Map.of(makePredicate("jdk.CPULoad"),
        ev -> Map.of("timestamp", ""+ ev.getStartTime(),
                     "user", ""+ ev.getDouble("jvmUser"),
                     "system", ""+ ev.getDouble("jvmSystem"),
                     "total", ""+ ev.getDouble("machineTotal")
                    ));
```

In this simple example, the `makePredicate()` method produces a `Predicate` object to test whether an incoming event is of interest, and then transforms it to a map of strings for output.

There are also lots of OSS tools available for handling JFR data. An interesting one is JFR Analytics by Gunnar Morling. It presents a SQL-like interface to querying JFR recording files and can be used with regular JDBC code.

Now that we understand the alternatives to sampling profilers and have seen JFR in context, it's time to return to the subject of how we use profiling within operational practice.

Operational Aspects of Profiling

Profilers are developer tools used to diagnose problems or understand the runtime behavior of applications at a low level. At the other end of the tooling spectrum are observability or operational monitoring tools. The latter exist to help a team visualize the current state of the system and determine whether the system is operating normally or is anomalous.

The space that these tools delineate is huge, and a single book simply cannot completely cover every tool in the space. Instead, we'll pick a few highlights to focus on.

These examples can serve as a starting point for you to begin to investigate the options and evaluate which is suitable for your applications. In observability and performance analysis, there is no easy path—you must engage fully and find the techniques that work for your specific domain of interest.

Using JFR As an Operational Tool

JFR has a long history, which is something of a double-edged sword. On the one hand, it has years of being battle-tested in production and is a best-in-class tool due to the deep integration with HotSpot.

However, it also comes from a much earlier time, when state of the art production profiling created a dump file (containing a lot of binary data and not human-readable) and then copied it back to a developer machine for offline analysis.

In our new world of cloud native applications, however, this may not be easy to achieve or very convenient. Simply put, we need new patterns and new methodology if JFR is to be a useful tool in the cloud native world.

One common operational pattern for using JFR is to start it in *ring buffer* configuration. This is usually done by starting the application with pre-configured JFR recording options that are passed to the `-XX:StartFlightRecording` switch, like this:

```
-XX:StartFlightRecording:disk=true,filename=/sandbox/service.jfr,maxage=4
settings=profile
```

This tells JFR that we want to keep events matching the "Profiling" profile that are up to four hours old in memory (and discarding events older than that as new events come in), and when we request a recording, to dump the current state of the buffer into `/sandbox/service.jfr`.

This allows an operator to dump a file when required and "go back in time" to after an incident has already started, provided the ring buffer is large enough. This can be an extremely useful technique during outage recovery.

It is, of course, necessary to allow for the additional memory that will be used to buffer events in the container, and this has a somewhat-subtle consequence that you should be aware of.

Note that in our example, we are using the `maxage` parameter to indicate how long we want events to be kept for. Thus, because JFR is event-based, the amount of data it generates pends on the JVM's activity (e.g., number of garbage collections) and does not necessarily have a hard cap.

In turn, this means it is possible, if an application is close to the container memory limits, that a burst of unexpected activity can cause the size of the JFR event buffer to overrun that limit, resulting in the container runtime or OOM-killer enforcing the limit and killing the application process.

For this reason, some teams prefer to specify the `maxsize` parameter instead, which provides a guarantee that JFR will not cause the container to be killed. However, this comes at the expense of not being 100% sure how much "look-back" time window is provided by the JFR ring buffer.

In addition, JFR will spool to a file on the filesystem, so care must also be taken to have sufficient free disk space (on bare metal and VMs) and also not to get evicted by a container runtime (Docker or K8s) for excessive I/O in what's supposed to be a stateless container.

More recent versions of Java, including both 17 and 21, also have a JFR Event Streaming capability. This is an API that lets programs receive callbacks for JFR events so the application (or an observability thread, perhaps in a Java agent) can respond to them as they occur. The key class here is the `RecordingStream`, which allows the developer to register an interest in particular event types and indicate a callback object to handle it.

This is much more suitable as the foundation for building an observability tool, but it suffers from the problem that Java 17+ has only a ~35% market share as of April 2024. Instead, other tooling capabilities have been developed.

Red Hat Cryostat

Cryostat is a JFR tool for containerized Java/JVM applications. It was originally developed by Red Hat and is natively supported on the OpenShift hybrid cloud platform but is available as an upstream OSS project that will work on any Kubernetes distribution.

Cryostat seeks to reduce the complexity of working with JFR recording files within a Kubernetes cluster. It provides the capability for users to remotely start, stop, retrieve, and analyze JFR event data.

Cryostat requires cert-manager, Operator Lifecycle Manager (OLM), and OperatorHub to deploy successfully. It offers features such as:

- Application topology view
- Grafana view (for metrics)
- Automated rules

- Notifications
- Smart triggers

In Figure 12-8, we can see the topology view that the general Kubernetes version of Cryostat provides.

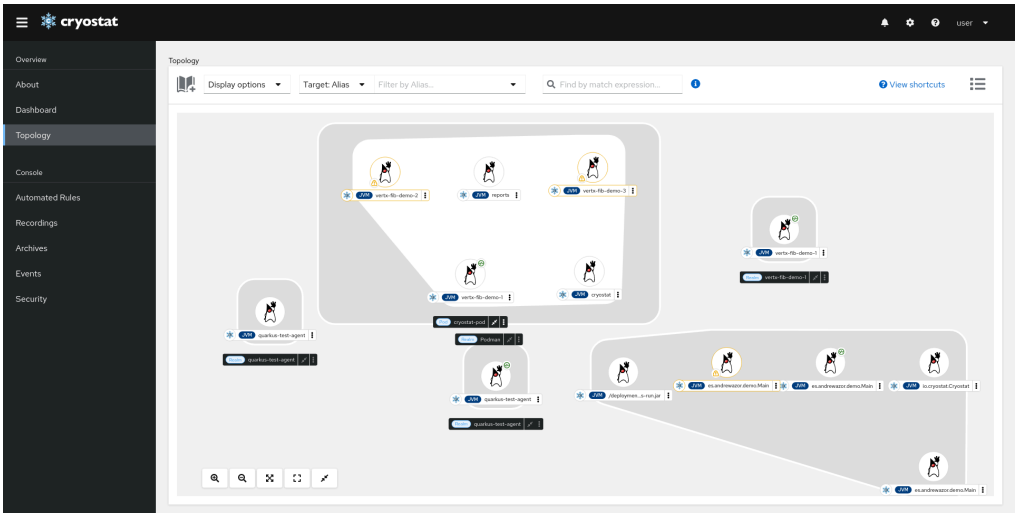

Figure 12-8. Cryostat topology view

The image shows a Kubernetes deployment of several pods, including both sample applications and the Cryostat pods themselves (another example of how observability services should themselves be observable).

Cryostat is an extremely useful tool for working with JFR, but a full discussion is outside the scope of this book. For completeness, in the proprietary tools space, both Datadog and New Relic (and possibly others) provide execution profilers that are based on JFR.

JFR and OTel Profiling

In "The Three Pillars", we met the three pillars of observability and discussed the possibility of profiling as a fourth pillar. This enhancement is being actively discussed within the OpenTelemetry working groups. For Java, multiple different profilers would potentially be usable as data sources for a future OTel profiling signal type.

JFR could be one of the data sources that provides the profiling signal, but there are some complications. For example, a practical OTel implementation of profiling would need to be based upon JFR Event Streaming due to OTel's time window restrictions, and so it would only work for Java 17+.

There is also the problem that a the time of writing (August 2024), there is no simple way to associate a trace ID with a JFR profiling sample. Work in this space is ongoing, not just in Java but in the other languages that are in scope for OpenTelemetry.

NOTE

The `opentelemetry-java-instrumentation` project ships an implementation of OpenTelemetry JVM metrics based on JFR, which defines a standard set of JVM metrics.

Overall, JFR should be seen as an important contributor and data source for the OTel ecosystem and as part of the overall shift toward open instrumentation, but it is not a complete execution profiling solution by itself.

Choosing a Profiler

There are several aspects to choosing a profiler that you need to take into account, such as:

- Do I need a tool for working interactively in a GUI?
- Is this tool for production profiling or more for dev/CI usage?
- How much time and sophistication can I afford to invest in setting up my profiler?
- Do I have any constraints on the overhead I can tolerate for my profiling solution?

In general, you can characterize the open source profilers like this:

- VisualVM is a GUI tool that is easy to deploy but requires direct JMX connection or a snapshot to work from.
- JMC is another, more sophisticated GUI tool, which can also use JFR (but only via dump files—streaming events are not supported).
- JFR is a low-overhead headless profiling engine built into the JVM—which can integrate with a variety of other tools (both for offline and more operational use cases).
- perf is very low level and concerned with hardware events on Linux—it is not Java specific and needs some additional bridging tools.
- Async Profiler builds on top of `perf_events` and provides a low-overhead solution that does not suffer from safepoint bias, but it probably has fewer available integrations than JFR.

Finally, there are proprietary commercial profilers available, such as JProfiler and YourKit. These were once clearly superior to the freely available tools, but that gap has arguably closed in recent years, although some teams still find value in these tools. We do not discuss them in this book, however, as our tooling focus is on open source products.

To conclude this chapter, let's move on from execution profiling and look at the other major form of profiling—memory.

Memory Profiling

Execution profiling is an important aspect of profiling, but it is not the only one! Many applications will also require some level of memory analysis, and here we will consider two primary types—allocation profiling and heap dump analysis.

Allocation Profiling

As we discussed in Chapter 4, one of the most important aspects of performance analysis is to consider the allocation behavior of the application. This leads to the discipline of *allocation profiling*, and several possible approaches could be applicable.

For example, we could use the Visitor pattern that tools like `jmap` rely upon.[7] In Figure 12-9, we can see the memory profiling view of VisualVM, which uses this approach to produce a histogram of memory used by each type.

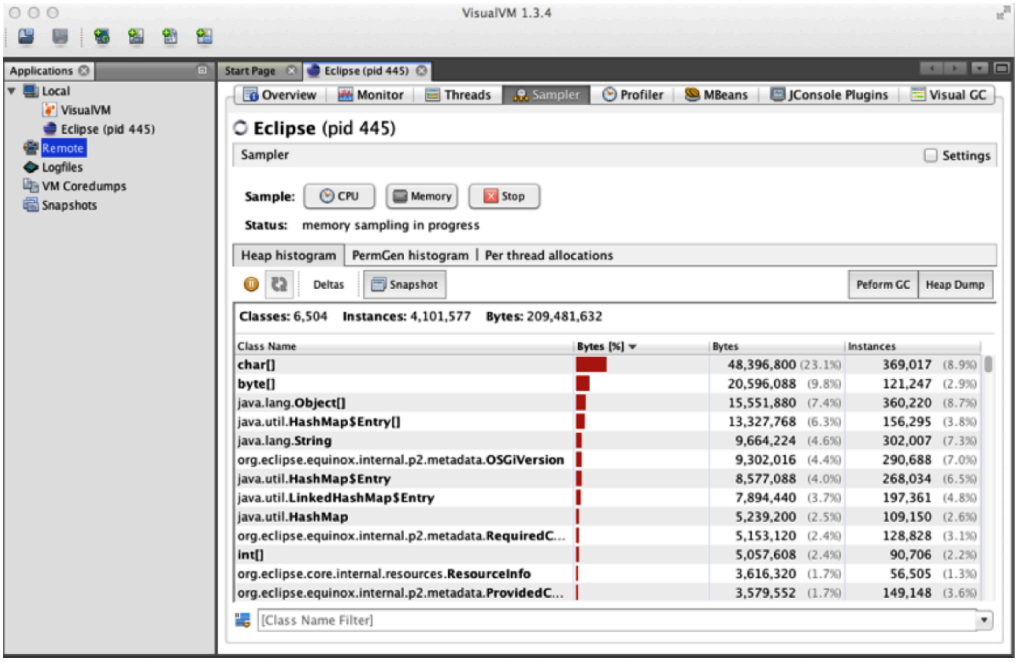

Figure 12-9. VisualVM memory profiler

This is a relatively simple view of memory, but there are a couple of things to point out here. First of all is that when producing this histogram, there are two options:

- A snapshot that's quick to produce but contains both live objects and garbage.

- An accurate snapshot, but one that needs an STW GC before being produced.

These two options correspond to the two command-line invocations: `jmap -histo` and `jmap -histo:live`.

Second, even a simple histogram can tell us a certain amount about an application's memory usage.

In most applications, strings are by far the most common data type. Inside a string is a reference to a `byte[]` (or a `char[]` in Java 8—the implementation changed with JEP 254), so we will expect to see at least as many `byte[]` objects as strings.

We also typically see other common objects, such as `HashMap` entries and `Object[]`. Business applications will also often see their domain objects appear in the list of most common objects—and this can provide a quick check by asking the question: "Is the observed amount of domain objects in the right ballpark for my application?"

Moving on from VisualVM, and slightly shifting focus from heap utilization to profiling GC, we can use the JMC tool to collect statistics that contain some values not available in the traditional Serviceability Agent (although the majority of counters presented are duplicates of the SA counters).

The advantage is that the cost for JFR to collect these values so that they can be displayed in JMC is much lower than it is with the SA. The JMC displays also provide greater flexibility to the performance engineer in terms of how the details are displayed.

Another approach to allocation profiling is to look at the TLABs, which you met in Chapter 4. In particular, JFR uses events to receive notifications when an object is allocated:

- In a TLAB (the `jdk.ObjectAllocationInNewTLAB` event)
- Outside of a TLAB (the "slow path," `jdk.ObjectAllocationOutsideTLAB` event)

This allows JFR to calculate how quickly memory is being allocated.

The Allocations view of JMC/JFR is capable of displaying the TLAB allocation view. Figure 12-10 shows a sample image of JMC's view of allocations.

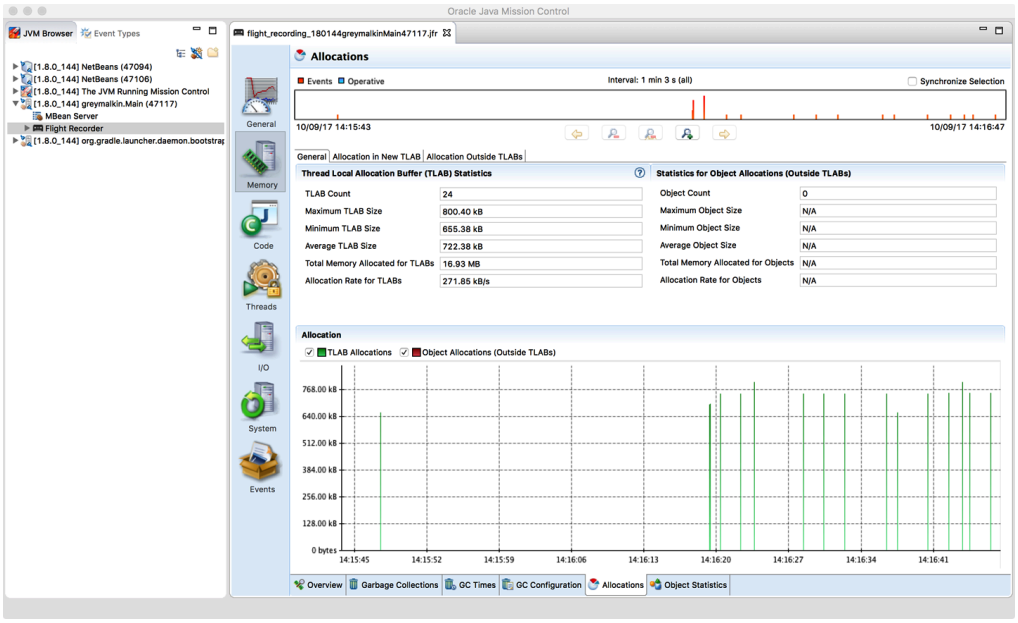

Figure 12-10. JMC Allocations profiling view

JFR also comes with events that can act as a form of memory leak profiler. It samples object allocations to track their lifetimes, using the `jdk.OldObjectSample` event, and over time, can provide suggestions for a starting point to investigate as a potential memory leak.

Heap Dumps

Another memory profiling technique is *heap dump analysis*. Unlike allocation profiling, this is an offline process, in which a snapshot of an entire heap is created and dumped to a file.

This can be achieved by the `jmap` command, for example, like this:

```
jmap -dump:live,format=b,file=heap.bin <pid>
```

This dump file can then be examined and analyzed by a separate tool to determine salient facts, such as the live set and numbers and types of objects, as well as the shape and structure of the object graph. These tools can either be batch mode or interactive.

With a heap dump loaded in an interactive tool, performance engineers can then traverse and analyze the snapshot of the heap at the time the heap dump was created. They will be able to see live objects and any objects that have died but not yet been collected.

One major drawback of heap dumps is their sheer size. A heap dump can frequently be 300%–400% the size of the memory being dumped, and for a production multigigabyte heap, this is substantial. Not only must the heap be written to disk, but for a real production use case, it must be re-

trieved over the network as well (which may not be easy for a containerized application).

Once retrieved, it must then be loaded on a workstation with sufficient resources (especially memory) to handle the dump without introducing excessive delays to the workflow. Working with large heap dumps on a machine that can't load the whole dump at once can be very painful, as the workstation pages parts of the dump file on and off disk.

Production of a heap file also requires the same tradeoff as we saw for the heap histogram—either garbage shows up alongside the live objects, or the process stops-the-world while the heap is traversed and the dump written out. In a modern cloud-based system, such an STW pause could cause the JVM process to appear down while a large heap was traversed, and this could result in the pod being killed.

Despite these difficulties, there are circumstances (such as difficult-to-isolate memory leaks) under which heap dumps can be useful. However, you should be aware of some of the limitations surrounding them, and avoid degrading or killing your production pods accidentally. Let's move on to discuss how to create and work with heap dumps.

As we saw in Figure 12-9, VisualVM can produce a heap dump. You can also use the tool to browse the contents of a heap dump file, but in practice, production heap dumps are somewhat unwieldy and difficult to work with in VisualVM. An alternative is the Eclipse Memory Analyzer (aka MAT), which is a standalone tool that can be used to analyze heap dumps.

A lot of the power of MAT comes from its ability to traverse the object graph and generate reports based on the structure of the heap. For example, MAT can be used to find potential memory leaks, analyze the top consumers and dominators of memory, and perform other memory-related tasks.

Figure 12-11 shows an example of MAT with several standard reports visible in tabs.

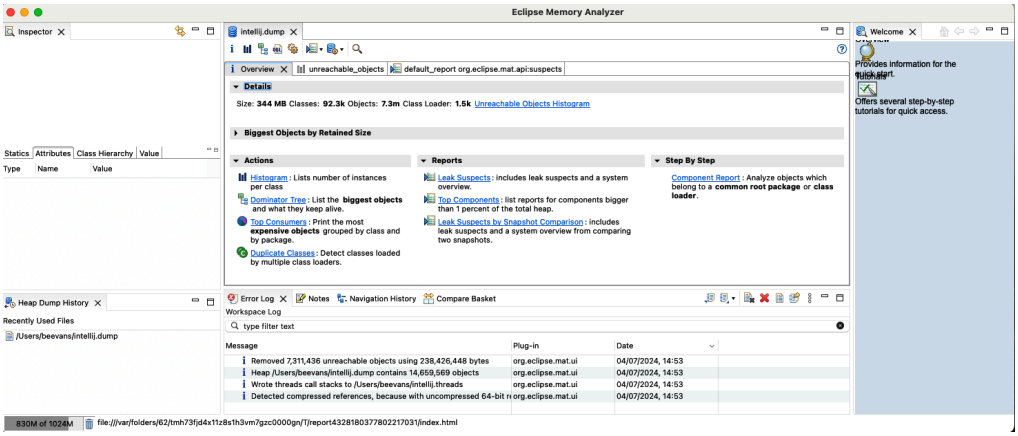

Figure 12-11. Example MAT view

In general, MAT is an advanced tool, and while the basic out-of-the-box reports are useful, the real power of MAT comes from spending some time learning how to use it effectively.

Allocation and heap profiling are of interest for the majority of applications that need to be profiled, and performance engineers are encouraged not to over-focus on execution profiling at the expense of memory.

As a final note, older texts may refer to the `hprof` heap profiling native agent. This was intended as a reference implementation for the JVMTI technology rather than as a production-grade profiler, and the documentation frequently points this out.

Despite this, quite a large number of developers started to consider `hprof` a suitable tool for actual use. For this reason, the `hprof` tool was removed in Java 9, although the ability to create heap dumps in the `hprof` format was retained in tools such as `jmap`. If your current toolchain uses `hprof`, then you should migrate to a supported tool such as MAT.

Summary

The subject of profiling is one that is often misunderstood by developers. Both execution and memory profiling are necessary techniques. However, it is very important that performance engineers understand what they are doing—and *why*. Simply using the tools blindly can produce completely inaccurate or irrelevant results and waste a lot of analysis time.

Profiling modern applications requires the use of tooling, and there are a wealth of options to choose from, including both commercial and open source options.

In the next chapter, we will move on from profiling and talk specifically about concurrency and how to use it efficiently in your Java apps. This will include key techniques that generalize to the distributed systems case, so they will be relevant to the cloud native case.

[1] "Structured Programming with go to Statements," *Computing Surveys*, vol. 6, no. 4 (December 1974): 268.

[2] This could include a full table scan or an ORM lazy-load issue, aka `N + 1` problem.

[3] Teams can also implement custom JFR events to add to their applications.

[4] It can be difficult to attach to a live production environment via JMX directly, so exfiltrating a JFR dump file is often an easier approach.

[5] This approach, based on `timer_create`, has also been adopted by the Go profiler as of 1.18.

[6] Humphrey Burchell, Octave Larose, Sophie Kaleba, and Stefan Marr, "Don't Trust Your Profiler: An Empirical Study on the Precision and Accuracy of Java Profilers," *MPLR 2023: Proceedings of the 20th ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes* (2023): 100–113, Association for Computing Machinery.

[7] The Visitor pattern is a classic design pattern that extracts an operation (in this case, memory analysis) into a separate class that can traverse the subcomponents of a composite object.